ragent Code Index Manual

This guide explains ragent's built-in **code index** — a tree-sitter-powered codebase indexing system that provides fast, structured search across all indexed source files. It covers the ten codeindex tools, the TUI slash commands, supported languages, the semantic code graph, and practical usage patterns.

> **Scope:** Code index tools, slash commands, graph analysis, language support, and troubleshooting. For the full tool catalog see `docs/howtos/tools/INDEX.md`. For TUI workflow see `docs/howtos/tutorial.md`.

Table of Contents

1. Overview
2. Enabling the Code Index
3. Supported Languages
4. Tool Reference
    - 4.1 codeindex_status
    - 4.2 codeindex_search
    - 4.3 codeindex_symbols
    - 4.4 codeindex_references
    - 4.5 codeindex_dependencies
    - 4.6 codeindex_reindex
    - 4.7 codeindex_explain
    - 4.8 codeindex_path
    - 4.9 codeindex_communities
    - 4.10 codeindex_godnodes
5. The Semantic Code Graph
6. System Instructions
7. Decision Flow: codeindex vs Grep
8. Slash Commands
9. Worked Examples
10. Troubleshooting
11. Related Documents

---

1. Overview

The code index (`ragent-codeindex` crate) provides structured search across your codebase. It uses **tree-sitter** parsers to extract symbols (functions, structs, enums, traits, etc.) from source files, stores them in a **SQLite** database, and builds a **Tantivy** full-text search index for fast keyword and symbol-name queries.

A background **file watcher** performs incremental updates — when a file changes on disk, only that file is re-parsed and re-indexed, keeping the index fresh without full re-scans.

Key capabilities:

- **Symbol search** — find functions, types, and modules by name or keyword
- **Reference lookup** — find every call site or usage of a named symbol
- **Dependency tracing** — query what a file imports and what depends on it

- **Semantic code graph** — build a directed edge graph between symbols, then run community detection, shortest-path, and hub-node analysis
- **Incremental updates** — a file watcher re-indexes changed files automatically
- **Busy-aware** — all tools return a `codeindex_busy` response instead of stalling when the index lock is held, so the agent loop is never blocked

---

2. Enabling the Code Index

The code index is controlled by the `codeindex` visibility switch (default `on`). Enable or disable it from the TUI:

```
/codeindex on
/codeindex off
```

Check status:

```
/codeindex show
```

When enabled, ragent opens the index database at `.ragent/codeindex/` within the project root, scans all source files, and starts the background file watcher. The first scan may take a few seconds on large codebases.

Enabling the code index registers the ten codeindex tools with the registry, so `/codeindex on` and `/codeindex off` are the only switch the family has.

---

3. Supported Languages

The code index ships tree-sitter parsers for these languages:

Language ID	Extensions	Notes
`rust`	`.rs`	Edition 2024 syntax
`python`	`.py`, `.pyi`	
`typescript`	`.ts`, `.tsx`	Also `tsx`, `javascript`, `jsx` variants of the same grammar
`javascript`	`.js`, `.jsx`	Shares the TS grammar
`go`	`.go`	
`c`	`.c`, `.h`	
`cpp`	`.cpp`, `.cc`, `.cxx`	
`java`	`.java`	
`terraform`	`.tf` (HCL)	
`openscad`	`.scad`	
`cmake`	`CMakeLists.txt`, `.cmake`	
`gradle`	`.gradle` (Groovy DSL)	
`gradle_kts`	`.gradle.kts` (Kotlin DSL)	
`maven`	`pom.xml`	

The scanner detects extensions for more languages (Kotlin, Ruby, Swift, and others), but files with no registered parser are not symbol-extracted.

Filter by language in `codeindex_search` and `codeindex_symbols` using the `language` parameter (e.g. `"rust"`).

Language-specific visibility:

```
/codeindex lang rust
```

This restricts indexing to a single language; a bare `/codeindex lang` lists the supported language IDs. Use `/codeindex show` to see which languages are currently active.

---

4. Tool Reference

All ten codeindex tools are **hardwired always-allowed** — they bypass the permission system entirely. Read tools use the `codeindex:read` permission category; `codeindex_reindex` uses `codeindex:write`.

When the index is not active, every tool returns a fallback message suggesting `Grep` or `Glob` as alternatives.

4.1 codeindex_status

Description: Show the current status and statistics of the codebase index — files indexed, symbols extracted, languages, index size, graph state (built / building / not built), and timestamps. The tool **never blocks**: when a background reindex or graph build holds the store lock it returns an immediate busy report built from the lock-free progress atomics instead of stalling or retrying.

Use cases: Check whether the index and graph are ready before running other codeindex tools; diagnose stale-index issues; observe live `done/total` progress while a reindex or graph build runs.

Schema:

```
{"type":"object","properties":{},"additionalProperties":false}
```

No parameters required.

Example:

```
codeindex_status
```

Output (idle):

```
## Code Index Status

Files indexed:  342
Total symbols:  5218
Total size:     128.5 KB
Languages:      rust (4210), python (1008)
FTS state:      built
Graph state:    built (3121 edges, 2870 nodes, 12 communities)
Last full:      2026-01-15T09:30:00Z
Last incremental: 2026-01-15T10:12:33Z
Index size:     45.2 KB
```

Output (busy — store lock held by a background operation):

```
## Code Index Status

Index:          busy (lock held by a background operation)
The store lock is held by another operation. Wait a moment and retry.
```

While a reindex runs, the busy report instead shows live progress:

```
Index:          busy (reindexing in progress)
Reindexing:     180/342 files
Graph building: 90/342 files
```

While only the graph build is running, the index itself stays available (the graph build holds the store lock only briefly for its snapshot/persist phases):

```
Graph building: 90/342 files
Index:          available (the graph build holds the store lock only briefly for its snapshot/per
```

The busy report is also returned in `metadata` as `{"busy": true, "error": "codeindex_busy", "reindexing": true, "reindex_done": 180, "reindex_total": 342, "graph_building": true, "graph_done": 90, "graph_total": 342}` so agents can poll build state programmatically.

4.2 codeindex_search

Description: Search the codebase index for symbols, functions, types, and documentation using full-text search with optional structured filters.

Use cases: Find where a function is defined; locate structs matching a pattern; search for symbols within a specific file path or language.

Schema:

```
{
  "type": "object",
  "properties": {
    "query": {"type":"string","description":"Symbol name, keyword, or phrase"},
    "kind": {"type":"string","enum":["function","struct","enum","trait","impl",
      "const","static","type_alias","module","macro","field","variant",
      "interface","class","method"]},
    "language": {"type":"string","description":"e.g. 'rust', 'python'"},
    "file_pattern": {"type":"string","description":"Path substring"},
    "max_results": {"type":"integer","description":"Default 20, max 100"}
  },
  "required": ["query"],
  "additionalProperties": false
}
```

Example:

```
codeindex_search query="parse_config" kind="function" language="rust" max_results=10
```

4.3 codeindex_symbols

Description: Query symbols from the codebase index with structured filters. All parameters are optional; combine them to narrow results.

Use cases: List all functions in a file; find all public structs; list every enum in a Rust crate.

Schema:

```
{
  "type": "object",
  "properties": {
    "name": {"type":"string","description":"Case-insensitive substring match"},
    "kind": {"type":"string","enum":["function","struct","enum","trait",
      "impl","const","static","type_alias","module","macro","field",
      "variant","interface","class","method"]},
    "file_path": {"type":"string","description":"Path substring"},
    "language": {"type":"string"},
    "visibility": {"type":"string","enum":["public","private","crate"]},
    "limit": {"type":"integer","description":"Default 50, max 200"}
  },
  "additionalProperties": false
}
```

Example:

```
codeindex_symbols kind="struct" language="rust" visibility="public" limit=50
codeindex_symbols file_path="src/parser" kind="function"
```

4.4 codeindex_references

Description: Find all references to a symbol by name across the indexed codebase. Returns file locations grouped by file, with reference kind (call, type, field_access).

Use cases: Find every call site of a function before refactoring; identify all type usages of a struct; locate field accesses for renaming.

Schema:

```
{
  "type": "object",
  "properties": {
    "symbol": {"type":"string","description":"Symbol name to look up"},
    "limit": {"type":"integer","description":"Default 50, max 200"}
  },
  "required": ["symbol"],
  "additionalProperties": false
}
```

Example:

```
codeindex_references symbol="parse_config" limit=100
```

4.5 codeindex_dependencies

Description: Query file-level dependencies from the code index. Tracks import/use edges between files.

Use cases: See what a file imports; find which files depend on a module before modifying it; trace the import chain.

Schema:

```
{
  "type": "object",
  "properties": {
    "path": {"type":"string","description":"Relative file path"},
    "direction": {"type":"string","enum":["imports","dependents"],
      "default":"imports"}
  },
  "required": ["path"],
  "additionalProperties": false
}
```

`imports` returns what the file uses; `dependents` returns what uses the file.

Example:

```
codeindex_dependencies path="src/main.rs" direction="dependents"
```

4.6 codeindex_reindex

Description: Trigger a full re-index of the codebase. Scans all files, extracts symbols, and updates the search index.

Use cases: After major file changes; when search results seem stale; after switching language filters.

Schema:

```
{"type":"object","properties":{},"additionalProperties":false}
```

No parameters required. This is the only codeindex tool with `codeindex:write` permission category.

Example:

```
codeindex_reindex
```

4.7 codeindex_explain

Description: Explain a symbol in the codebase graph — show its node metadata (source file, line, community, degree) and its incoming/outgoing edges with kind and confidence tags. Limited to the top 50 connections.

Use cases: Understand a symbol's role in the architecture; see what calls it and what it calls; identify hub symbols.

Schema:

```
{
  "type": "object",
  "properties": {
    "symbol": {"type":"string","description":"Symbol name to explain"}
  },
  "required": ["symbol"],
  "additionalProperties": false
}
```

Example:

```
codeindex_explain symbol="EventBus"
```

Output:

```
## Explain: EventBus

Node: `EventBus` in `src/event_bus.rs:42` (community 3, degree 18)

### Incoming edges (callers)
  parse_handler --call--> EventBus
  session_processor --type--> EventBus

### Outgoing edges (callees)
  EventBus --call--> publish
  EventBus --call--> subscribe
```

4.8 codeindex_path

Description: Find the shortest path (by hop count) between two symbols in the codebase graph, displaying each hop as `A --kind--> B` with confidence tags.

Use cases: Trace how two unrelated symbols are connected; understand the dependency chain between modules; find the coupling distance.

Name resolution prefers exact matches over the underlying substring query and ranks definition kinds (struct/function/trait/class/enum/interface) above impl/module containers. This ranking applies to **both** `codeindex_path` and `codeindex_explain`, so a trait like `CachedSessionProcessor` no longer shadows the struct `SessionProcessor`.

Schema:

```
{
  "type": "object",
  "properties": {
    "from": {"type":"string","description":"Source symbol name"},
    "to": {"type":"string","description":"Target symbol name"}
  },
  "required": ["from","to"],
  "additionalProperties": false
}
```

Example:

```
codeindex_path from="main" to="parse_config"
```

Output:

```
## Path: main --> parse_config

main --call--> run_app
run_app --call--> init_config
init_config --call--> parse_config

Hops: 3
```

4.9 codeindex_communities

Description: Run community detection over the codebase graph and display each detected community with its auto-generated label and member count. Uses label-propagation algorithm.

Use cases: Identify logical modules; spot unexpected coupling between distant communities; understand high-level codebase structure.

Schema:

```
{"type":"object","properties":{},"additionalProperties":false}
```

No parameters. Requires the graph to be built (`/codeindex graph build`).

Example:

```
codeindex_communities
```

Output:

```
## Communities

| Community | Label | Members |
|-----------|-------|---------|
| 0 | parser-core | 42 |
| 1 | llm-providers | 28 |
| 2 | tui-rendering | 35 |
| 3 | tool-registry | 19 |
```

4.10 codeindex_godnodes

Description: Display the top-N most-connected symbols (highest degree) in the codebase graph with their names, source files, and edge counts.

Use cases: Identify architectural bottlenecks; find hub functions that everything depends on; spot candidates for refactoring.

Schema:

```
{
  "type": "object",
  "properties": {
    "n": {"type":"integer","minimum":1,
      "description":"Max god-nodes to return (default 10, max 100)"}
  },
  "additionalProperties": false
}
```

Example:

```
codeindex_godnodes n=20
```

Output:

```
## God Nodes (Top Most-Connected Symbols)

| # | Symbol | Source File | Degree |
|---|--------|-------------|--------|
| 1 | `EventBus` | `src/event_bus.rs` | 18 |
| 2 | `Config` | `src/config.rs` | 15 |
| 3 | `ToolContext` | `src/tool/mod.rs` | 12 |
```

---

5. The Semantic Code Graph

The code index can build a **semantic code graph** — a directed graph where nodes are symbols and edges represent relationships (calls, type references, field accesses, imports). Edge confidence is tracked as a score.

Build the graph from the TUI:

```
/codeindex graph build
```

This derives edges from the indexed symbols and stores them in the SQLite database. The build runs on a dedicated OS thread (`codeindex-graph-build`), so the TUI stays responsive: the command returns immediately with a `[wait]` status, the status bar animates a `graph` busy tag, and the completion message arrives when the build finishes. The build uses a phased lock discipline — a brief read-only snapshot of the derivation inputs, a lock-free in-memory derivation, and a brief single-transaction persist — so search and the other store readers remain available while the graph builds. A double-build guard refuses overlapping builds. `/codeindex reindex` also runs in the background the same way (with an `idx` status-bar tag).

The graph tools (`codeindex_explain`, `codeindex_path`, `codeindex_communities`, `codeindex_godnodes`) all require the graph to be built first. While a build is in flight, `/codeindex show` reports `**Graph:** building...` with per-file `done/total` progress, and `codeindex_status` reports `graph_state: "building"`.

Export the graph for external analysis:

```
/codeindex graph export
```

Filter the graph to a single language:

```
/codeindex graph lang rust
```

Graph statistics appear in `/codeindex show` output, including edge count and whether the graph has been built.

---

6. System Instructions

The agent receives the following instruction in its system prompt when the code index is active:

> **MANDATORY — You MUST use codeindex tools instead of Grep for code symbol queries.** When the index is active, `Grep` is the WRONG choice for finding functions, types, structs, enums, traits, or any named code entity. The index is faster, returns structured results with file/line/signature, and understands symbol kinds.

This instruction shapes the agent's tool-selection behaviour: for any query about a named code entity, the agent reaches for `codeindex_search` or `codeindex_symbols` rather than `Grep`.

---

7. Decision Flow: codeindex vs Grep

Query type	Use
"Where is function X defined?"	`codeindex_search`
"Find all structs matching Y"	`codeindex_symbols` with `kind=struct`
"Who calls function Z?"	`codeindex_references`
"What does file A import?"	`codeindex_dependencies`
"List all functions in file B"	`codeindex_symbols` with `file_path`
"Is the index working?"	`codeindex_status`
"Re-index after bulk edits"	`codeindex_reindex`
Find TODO/FIXME comments	`Grep`
Search config files or markdown	`Grep`
Search for arbitrary text patterns	`Grep`

Rule of thumb: If you are looking for a named code entity (function, type, variable, import), use codeindex. If you are searching for a text pattern that is NOT a code symbol, use `Grep`.

---

8. Slash Commands

All codeindex slash commands are available in the TUI:

```
/codeindex on              Enable codebase indexing
/codeindex off             Disable codebase indexing
/codeindex show            Show index and graph status & statistics
/codeindex status          Alias of `show`
/codeindex lang <language> Set language filter (e.g. rust); bare `lang` lists supported languages
/codeindex reindex         Trigger a full re-index
```

```
/codeindex rebuild          Rebuild the FTS index
/codeindex graph build      Build the semantic edge graph
/codeindex graph export     Export graph.json and GRAPH_REPORT.md
/codeindex graph lang <l>   Filter the graph to a single language
/codeindex explain <sym>    Explain a symbol's connections
/codeindex path <A> <B>     Shortest path between two symbols
/codeindex communities      List detected communities
/codeindex godnodes         List high-degree hub symbols
/codeindex skillgen         Install the graphify skill to ~/.ragent/skills/
/codeindex help             Show available sub-commands
```

---

9. Worked Examples

Example 1: Find a function definition

`codeindex_search query="parse_config" kind="function"`

Returns the file, line, and signature of every `parse_config` function.

Example 2: Find all call sites before refactoring

`codeindex_references symbol="parse_config" limit=100`

Returns every file and line that references `parse_config`, grouped by file.

Example 3: Trace what depends on a file

`codeindex_dependencies path="src/config.rs" direction="dependents"`

Returns every file that imports or uses `src/config.rs`.

Example 4: Identify architectural hubs

`codeindex_godnodes n=15`

Returns the 15 most-connected symbols — likely the central types and functions in your architecture.

Example 5: Understand two symbols' connection

`codeindex_path from="main" to="publish"`

Returns the shortest call-chain path between `main` and `publish`.

Example 6: Map the codebase into modules

`codeindex_communities`

Returns a table of detected communities with auto-generated labels, revealing the logical module structure of the codebase.

---

10. Troubleshooting

“Code index is not active”

The index is disabled. Enable it:

```
/codeindex on
```

“No graph data available”

The semantic graph has not been built. Build it:

```
/codeindex graph build
```

“codeindex_busy” response

The index is temporarily locked by a background operation (a reindex or a graph build). The tool returns immediately with a busy message instead of stalling; the busy report includes live `done/total` progress for the reindex and graph phases. The TUI status bar also animates `idx` / `graph` busy tags for the same conditions. Wait for the build to finish or check progress:

```
/codeindex show
```

Search results are stale

Trigger a full re-index:

```
/codeindex reindex
```

Or rebuild the FTS index:

```
/codeindex rebuild
```

“FTS index is empty”

The full-text search index has no documents. Run:

```
/codeindex rebuild
```

---

11. Related Documents

Document	Covers
`docs/howtos/tools/INDEX.md`	Full tool catalog (149 registered static tools in the default build, 150 with a drivable headless browser engine present, + 1 dynamic tool, split by category)
`docs/howtos/tutorial.md`	End-to-end TUI workflow tutorial
`docs/howtos/custom-agents.md`	Custom agent profiles
`TUI-QUICKSTART.md`	TUI layout, panels, startup options